

# Sistema de Recuperación de Información sobre Tecnología y Software

*Alex Moreno Rodriguez*  
*Jery Rodríguez Fernández*

Sistemas de Recuperación de Información — Curso 2025-2026

**Abstract.** Este documento presenta el diseño e implementación completa de un Sistema de Recuperación de Información (SRI) orientado al dominio de tecnología y software. El sistema integra de forma coherente: un crawler bilingüe (inglés/español) que recopiló 1.361 documentos de 248 dominios únicos, un índice invertido Patricia Trie con 59.427 términos únicos, un modelo recuperador Vectorial Generalizado (GVSM) con fusión BM25 y búsqueda vectorial ChromaDB, un módulo RAG basado en el LLM llama-3.1-70b de Cerebras con fallback extractivo, un módulo de búsqueda web automático (DuckDuckGo) y una interfaz visual React/Vite bilingüe. El sistema completo puede desplegarse mediante Docker con dos servicios independientes: API Flask (puerto 5000) y frontend Vite (puerto 5173).

---

## 1 Introducción

Los sistemas de recuperación de información (SRI) constituyen una disciplina fundamental de las ciencias de la computación cuyo objetivo es encontrar material —generalmente documentos— que satisfaga una necesidad de información dentro de colecciones de gran tamaño. El presente proyecto implementa un SRI completo orientado al dominio de tecnología y software, integrando de forma modular todos los componentes requeridos: adquisición, indexación, recuperación, generación aumentada (RAG), búsqueda web, posicionamiento e interfaz visual.

El dominio de tecnología y software fue seleccionado por su riqueza terminológica técnica, la alta correlación entre términos (p.ej., 'python' y 'django', 'kubernetes' y 'docker'), la disponibilidad de fuentes de calidad en la web, y la cobertura bilingüe natural (inglés/español) que lo hace especialmente apropiado para explorar recuperación multilingüe.

El modelo de recuperación seleccionado es el Modelo Vectorial Generalizado (GVSM), propuesto por Wong, Ziarko y Wong (1985) [1], cuya capacidad de capturar correlaciones entre términos técnicos lo distingue del VSM clásico y lo hace particularmente adecuado para este dominio. Este modelo se complementa con BM25 como señal léxica y ChromaDB como capa vectorial para conformar un sistema de ranking híbrido.

## 2 Dominio Temático y Fuentes de Datos

### 2.1 Justificación del Dominio

El dominio de tecnología y software fue seleccionado por los siguientes criterios:

- Disponibilidad de fuentes de alta calidad y actualizadas: documentación oficial de lenguajes, frameworks, blogs de ingeniería de grandes empresas tecnológicas y publicaciones académicas.

- Riqueza terminológica técnica que favorece la evaluación de modelos de recuperación avanzados con correlaciones de co-ocurrencia densas y significativas.
- Cobertura bilingüe natural (inglés/español): el corpus resultante contiene un 80,21 % de documentos en inglés y un 19,79 % en español, reflejando la distribución real de contenido técnico en la web.
- Relevancia práctica: los usuarios con necesidades de información técnica formulan consultas bien definidas, lo que facilita la evaluación cualitativa del sistema.

## 2.2 Fuentes de Datos

Las semillas de crawling fueron seleccionadas para cubrir las siguientes categorías del dominio:

- Documentación oficial de lenguajes: Python (EN/ES), Rust, Go, Java, .NET (EN/ES), Kotlin.
- Frameworks y herramientas: React, Angular, Vue, Node.js, Django (EN/ES), Flask.
- Blogs de ingeniería: Netflix Tech Blog, Facebook Engineering, Google Open Source, AWS Architecture Blog, Stack Overflow Blog.
- Referentes técnicos: Martin Fowler, CSS-Tricks, Smashing Magazine.
- Recursos académicos y estándares: arXiv (CS reciente), W3C TR, RFC Editor, Semantic Scholar.

## 3 Módulo de Adquisición de Datos

### 3.1 Arquitectura del Crawler

El módulo de adquisición está organizado en el paquete `crawler_engine/` con los siguientes componentes reutilizables e independientes:

- `SimpleHTMLExtractor (extractor.py)`: parser HTML basado en `HTMLParser` de la biblioteca estándar que extrae título, texto visible, enlaces salientes y metadatos de idioma, ignorando etiquetas `script`, `style`, `noscript` y `svg`.
- `score_relevance (relevance.py)`: función de puntuación basada en presencia de más de 40 palabras clave técnicas en inglés y español, aplicada sobre título, primeros 5.000 caracteres del texto y URL.
- `canonicalize_url (utils.py)`: normalización de URLs para deduplicación, eliminando query strings, fragmentos y barras finales redundantes.
- `CrawlConfig` y `CrawlState (models.py)`: dataclasses que encapsulan la configuración de ejecución y el estado mutable del crawler, facilitando su parametrización y extensión.
- Gestión de robots.txt mediante `RobotFileParser` con caché por dominio raíz para evitar peticiones repetidas.

### 3.2 Proceso de Crawling

El crawler opera con una frontera de prioridad (min-heap) donde cada `QueueItem` contiene prioridad (negativo del puntaje de relevancia del padre), profundidad actual y URL. Por cada URL extraída de la frontera se aplica el siguiente flujo:

- Verificación de límites: URLs ya visitadas, límite por dominio (default: 120), profundidad máxima (default: 2).
- Validación de robots.txt: respeto a la directiva `Disallow` para el agente `IRS-TechCrawler/1.1 (+academic-project)`.

- Throttling por dominio: espera configurable (default: 1,0 s) entre accesos al mismo dominio para no sobrecargar servidores.
- Extracción de contenido: título, texto plano, enlaces salientes e idioma detectado mediante SimpleHTMLExtractor.
- Filtrado de relevancia: documentos con menos de 300 caracteres o con puntaje de relevancia nulo son descartados.
- Deduplicación por contenido: hash SHA-256 del texto para evitar indexar el mismo contenido bajo distintas URLs.
- Almacenamiento en JSONL con campos: doc\_id, url, title, domain, crawl\_date, text, language, source\_type, out\_links, relevance\_score, depth, parent\_url.

### 3.3 Detección de Idioma

La detección de idioma opera en cascada con cuatro niveles de confianza decreciente:

- Metaetiquetas HTML: atributo lang del elemento <html>, meta http-equiv='content-language', og:locale.
- Cabecera HTTP Content-Language retornada por el servidor.
- Patrones en la URL: subdominios tipo es.site.com o segmentos de ruta /en/ y /es/.
- Análisis heurístico de stopwords: comparación del vocabulario del texto con listas de stopwords en inglés y español. Se requieren al menos 40 tokens y una diferencia de puntuación significativa para emitir un resultado.

### 3.4 Políticas de Crawling Responsable

- Respeto a robots.txt con caché de parsers por dominio raíz.
- User-Agent identificado: IRS-TechCrawler/1.1 (+academic-project).
- Delay configurable entre accesos al mismo dominio (default: 1,0 s).
- Límite de páginas por dominio (default: 120) para evitar sobrecarga de un único servidor.
- Extensiones binarias descartadas sin descarga: .pdf, .zip, imágenes, videos, documentos Office y ejecutables.

### 3.5 Estadísticas del Corpus

Tras la ejecución del crawler sobre las semillas definidas con profundidad máxima 2, el corpus recopilado presenta las siguientes características:

| Métrica                                    | Valor                       |
|--------------------------------------------|-----------------------------|
| Total de documentos recopilados            | 1.361                       |
| Tamaño promedio por documento (caracteres) | 14.351,15                   |
| Tamaño total del corpus                    | 40.072.093 bytes (~38,2 MB) |
| Número de dominios únicos                  | 248                         |
| Proporción de documentos en inglés         | 80,21 %                     |
| Proporción de documentos en español        | 19,79 %                     |
| Profundidad máxima de crawl utilizada      | 2                           |
| Términos únicos en el índice               | 59.427                      |

El corpus presenta una distribución bilingüe coherente con la disponibilidad real de contenido técnico en la web, donde el inglés domina ampliamente. Los 248 dominios únicos reflejan una cobertura diversa del dominio, evitando la concentración excesiva en un único servidor gracias al límite por dominio configurado.

## 4 Módulo de Indexación

### 4.1 Preprocesamiento de Texto

El preprocesamiento se implementa en la clase Index (indexer.py) utilizando spaCy con el modelo en\_core\_web\_sm. El pipeline de tokenización aplica los siguientes pasos en orden:

- Lematización mediante el modelo spaCy seleccionado (inglés por defecto, español disponible como es\_core\_news\_sm).
- Conversión a minúsculas.
- Normalización Unicode NFKD con eliminación de diacríticos.
- Filtrado de tokens de puntuación, espacios en blanco y marcadores especiales (-PRON-).
- Eliminación de tokens con longitud  $\leq 2$  caracteres y tokens no alfabéticos.
- Remoción de stopwords mediante el flag is\_stop de spaCy y listas personalizadas en inglés (37 términos) y español (82 términos).

El resultado es un vocabulario de 59.427 términos únicos sobre 1.361 documentos, con una densidad terminológica que favorece la construcción de correlaciones de co-ocurrencia significativas para el GVSM.

### 4.2 Índice Invertido: Patricia Trie

El índice invertido se implementa como un Patricia Trie comprimido (clase PatriciaTrie, indexer.py). Esta estructura almacena términos como caminos comprimidos desde la raíz, donde cada arco representa un prefijo compartido en lugar de un único carácter, reduciendo el uso de memoria respecto a un trie estándar.

Las operaciones implementadas son:

- insert(word, doc\_id): inserción con compresión de arcos compartidos y actualización de posting lists. Soporta división de arcos (edge splitting) cuando solo hay coincidencia parcial.
- search(word): búsqueda exacta que retorna la lista de doc\_ids asociada al término.
- get\_parcial\_AND(terms, min\_match, max\_candidates): recuperación parcial que retorna documentos con al menos min\_match términos coincidentes, ordenados por frecuencia de matches descendente. Implementado con un contador por documento para eficiencia.
- Persistencia en formato JSON flat con nodos indexados numéricamente (sin referencias de puntero), compatible con serialización estándar.
- Carga incremental: al iniciar, el trie se sincroniza automáticamente con los últimos documentos del JSONL (lectura en orden inverso con FileReadBackwards) sin reconstruir el índice completo.

### 4.3 Índice de Co-ocurrencia

La clase `CoOccurrenceIndex` (`engine.py`) mantiene estadísticas de frecuencia documental (`df`) y co-ocurrencia de pares de términos en formato de matriz dispersa CSR (`scipy.sparse.csr_matrix`). Estas estadísticas alimentan la construcción de la matriz de correlación del GVSM.

- Solo el triángulo superior de la matriz se almacena para evitar duplicación de datos.
- Umbral mínimo de co-ocurrencia configurable (default: 3) para filtrar pares espurios con pocas apariciones conjuntas.
- Serialización a JSON con formato sparse (arrays separados de filas, columnas y valores) compatible con `scipy.sparse.csr_matrix`.
- Reconstrucción automática si el número de documentos del corpus cambia respecto al caché persistido en `data/processed/cooccurrence_index.json`.

## 5 Módulo Recuperador: Modelo Vectorial Generalizado

### 5.1 Selección y Justificación del Modelo

Se implementó el Modelo Vectorial Generalizado (Generalized Vector Space Model, GVSM), propuesto por Wong, Ziarko y Wong (1985) [1]. Su selección sobre otras opciones disponibles se justifica por las siguientes razones:

- El dominio tecnológico presenta alta correlación entre términos técnicos (p.ej., 'python' co-ocurre frecuentemente con 'flask', 'django', 'pandas'). El GVSM captura estas correlaciones explícitamente mediante la matriz de correlación término-término, a diferencia del VSM clásico que asume ortogonalidad.
- El corpus de 1.361 documentos y 59.427 términos únicos produce una matriz de co-ocurrencia dispersa que puede representarse eficientemente con `scipy.sparse.csr_matrix`, haciendo viable la operación  $q^T \cdot A_{\text{sub}} \cdot d$  en tiempo razonable.
- El GVSM permite recuperar documentos relevantes aunque no contengan exactamente los términos de la consulta, siempre que sus términos estén correlacionados con los de la consulta. Esto es especialmente valioso en un dominio donde existen múltiples sinónimos técnicos.

### 5.2 Definición Formal

El GVSM extiende el Modelo Vectorial Estándar representando documentos y consultas en un espacio vectorial donde los ejes no son ortogonales, sino que tienen correlaciones definidas por la co-ocurrencia documental de los términos. La similitud entre una consulta  $q$  y un documento  $d$  se calcula como:

$$\text{sim}(q, d) = (\sum_i \sum_j w_{iq} \cdot w_{jd} \cdot \text{corr}(t_i, t_j)) / (\|q\| \cdot \|d\|)$$

Donde:

- $w_{iq}$  y  $w_{jd}$  son los pesos TF-IDF normalizados de los términos  $t_i$  y  $t_j$  en la consulta y el documento, respectivamente.
- $\text{corr}(t_i, t_j)$  es la correlación coseno entre los términos  $t_i$  y  $t_j$ , calculada sobre sus vectores de frecuencia documental en la co-ocurrencia.
- $\|q\|$  y  $\|d\|$  son las normas euclidianas de los vectores de peso de consulta y documento.

### 5.3 Implementación

La implementación en la clase `GeneralizedVectorSpaceModel` (`engine.py`) incluye:

- IDF suavizado:  $\log((N+1)/(df+1)) + 1$ , donde N es el total de documentos (1.361) y df la frecuencia documental del término.
- TF normalizado por el TF máximo en el documento o la consulta.
- Matriz de correlación término-término construida como matriz dispersa simétrica a partir del índice de co-ocurrencia, con umbral `min_cooc=3`. Se invalida automáticamente cuando cambia el vocabulario o el umbral.
- Similitud GVSM implementada con operaciones matriciales dispersas de `scipy`:  $\text{score} = q^T \cdot A_{\text{sub}} \cdot d$ , donde  $A_{\text{sub}}$  es la submatriz de correlación indexada por los términos de la consulta (filas) y del documento (columnas).
- Pre-cómputo de vectores de todos los documentos al inicializar el motor para reducir latencia en tiempo de consulta.

## 5.4 Fusión Léxico-Semántica

El recuperador combina tres señales mediante fusión lineal con normalización min-max por conjunto de candidatos:

$$\text{score}(d) = (1 - \lambda) \cdot [\alpha \cdot \text{sem\_norm}(d) + (1-\alpha) \cdot \text{lex\_norm}(d)] + \lambda \cdot \text{chroma\_norm}(d)$$

- `sem_norm(d)`: puntuación GVSM normalizada al rango  $[0,1]$ . Peso  $\alpha = 0,6$ .
- `lex_norm(d)`: puntuación BM25 normalizada. BM25 usa  $k1 = 1,2$  y  $b = 0,75$ , con boost de  $2\times$  para coincidencias en el título del documento. Peso  $(1-\alpha) = 0,4$ .
- `chroma_norm(d)`: similitud coseno ChromaDB normalizada. Peso  $\lambda = 0,75$ .

## 5.5 Recuperación Híbrida de Candidatos

El conjunto de candidatos para el ranking se forma como la unión de dos fuentes:

- Patricia Trie (`get_parcial_AND`): candidatos que coinciden con al menos `min_match` términos de la consulta. Para consultas de un término, `min_match = 1`; para dos o más, `min_match = 2`.
- ChromaDB (búsqueda vectorial): top-k documentos por similitud coseno en el espacio de embeddings hash de dimensión 384.

Si la recuperación estricta (`min_match = 2`) no produce candidatos, se relaja a `min_match = 1` automáticamente. Para consultas de un solo término, los candidatos de ChromaDB se filtran adicionalmente por coincidencia léxica en título, texto o URL, evitando resultados puramente vectoriales sin relación léxica con la consulta.

## 5.6 Fuente Bibliográfica

[1] Wong, S.K.M., Ziarko, W., Wong, P.C.N. (1985). Generalized vector space model in information retrieval. In Proceedings of the 8th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 18–25. ACM, New York.

# 6 Base de Datos Vectorial

## 6.1 Tecnología: ChromaDB

La base de datos vectorial se implementa con ChromaDB (`chroma_store.py`), una base de datos vectorial de código abierto que provee almacenamiento persistente y búsqueda por similitud coseno mediante índices HNSW (Hierarchical Navigable Small World) [4]. La colección se configura con espacio de similitud coseno (`hnsw:space = cosine`) y se almacena en `data/processed/chroma_db/`.

## 6.2 Generación de Embeddings

Los embeddings se generan mediante la función `ChromaVectorStore._embed_text()`, que aplica una proyección hash determinista sobre los tokens del documento. Para cada token se calcula un resumen SHA-1, se usa los primeros 4 bytes para determinar la posición en un vector de dimensión 384, y el byte 5 para determinar el signo ( $\pm 1$ ). El vector resultante se normaliza a norma unitaria. Este enfoque garantiza reproducibilidad y ausencia de dependencias externas de modelos de lenguaje pre-entrenados, con la posibilidad de reemplazarse por embeddings semánticos en versiones futuras.

## 6.3 Integración con el Motor de Búsqueda

Al inicializar el motor, todos los documentos del corpus se upsertean en la colección ChromaDB mediante `upsert_documents()`. Durante la búsqueda, la consulta se vectoriza con `_embed_text()` y se ejecuta una query vectorial que retorna pares (`doc_id`, `distancia_coseno`). La distancia se convierte a similitud como `score = 1 - distancia`. Los documentos ingresados dinámicamente vía búsqueda web también se upsertean en ChromaDB para estar disponibles en consultas posteriores.

# 7 Módulo RAG

## 7.1 Arquitectura General

El módulo RAG (`rag_engine/`) implementa un pipeline de Retrieval-Augmented Generation [2] que integra el recuperador GVSM con el LLM llama-3.1-70b de Cerebras. La arquitectura es modular con separación clara de responsabilidades entre sus cinco componentes:

- `RAGConfig (config.py)`: configuración centralizada cargada desde variables de entorno (`.env`). Parámetros principales: modelo (llama-3.1-70b), temperatura (0,7), `max_tokens` (1.200), `timeout` (30 s), `max_retries` (3), `retry_backoff_factor` (2,0).
- `CerebrasClient (cerebras_client.py)`: cliente de la API de Cerebras Cloud con lógica de retry exponencial. Usa el endpoint `chat.completions.create()` y extrae la respuesta de `choices[0].message.content`.
- `CerebrasRAGGenerator (generator.py)`: orquestador principal que coordina la generación LLM, el análisis del flag de suficiencia y la activación del fallback.
- `ImprovedRAGGenerator (fallback_generator.py)`: generador extractivo de respaldo basado en scoring de sentencias por overlap de tokens con los términos de la consulta.
- `RAGPipeline (rag.py)`: interfaz pública que mantiene compatibilidad con el servidor Flask y orquesta la interacción con el motor de búsqueda y el módulo de búsqueda web.

## 7.2 Proceso de Generación

1. Recuperación: `retriever.search(query, top_k=5)` obtiene los documentos más relevantes mediante el motor GVSM híbrido.
2. Construcción del prompt: `build_rag_prompt()` construye un prompt estructurado con la consulta y fragmentos de los documentos recuperados. Presupuesto total de 2.400 caracteres (~480 por documento), con limpieza de etiquetas HTML residuales.
3. Generación LLM: `CerebrasClient` envía el prompt a la API de Cerebras (llama-3.1-70b, temperatura 0,7, máximo 1.200 tokens) con hasta 3 reintentos con espera exponencial (1 s, 2 s, 4 s).
4. Análisis de suficiencia: `parse_sufficient_flag()` extrae el objeto JSON de la primera línea de la respuesta (`{"sufficient": bool}`) para determinar si los documentos recuperados son adecuados para responder la consulta.

5. Truncado: `truncate_answer()` respeta límites de oraciones (busca el último punto seguido de espacio en el 80 % final del texto) antes de aplicar el corte por caracteres.
6. Atribución: `extract_used_doc_ids()` detecta qué documentos fueron citados implícitamente en la respuesta por overlap de palabras significativas ( $\text{longitud} > 3$ ) del título con el texto generado.

### 7.3 Prompt Engineering

El prompt está redactado en español con las siguientes directivas explícitas al modelo:

- Responder únicamente con información presente en los documentos recuperados.
- Indicar explícitamente si los documentos no son suficientes para responder la consulta.
- No inventar datos ni contenido no respaldado por los documentos.
- Responder en el mismo idioma de la consulta del usuario.
- Mantener la respuesta entre 300 y 1.200 caracteres.
- Emitir en la primera línea un objeto JSON: `{"sufficient": true/false}`, seguido de una línea en blanco y luego la respuesta en texto natural.

### 7.4 Generador Extractivo de Fallback

Si Cerebras falla tras 3 reintentos, ImprovedRAGGenerator produce una respuesta extractiva con las siguientes heurísticas:

- División del texto de cada documento en oraciones mediante expresiones regulares sobre signos de puntuación (.!?) y saltos de línea.
- Puntuación de cada oración:  $\text{score} = \text{overlap\_ratio} \times (1 + \min(1, \text{doc\_score} \times 0,5))$ , donde  $\text{overlap\_ratio} = \text{términos\_consulta\_en\_oración} / \max(5, \text{len\_tokens\_oración})$ .
- Selección de las mejores oraciones respetando diversidad de fuentes (máximo 2 por documento,  $\text{max\_sentences} = 6$ ) y presupuesto de 1.200 caracteres.
- Fallback final: fragmento de los primeros 1.200 caracteres del primer documento si no se extraen oraciones relevantes.

### 7.5 Integración RAG–Búsqueda Web

Cuando el LLM indica `sufficient = false`, RAGPipeline activa WebSearchPipeline, ingesta los documentos obtenidos en el motor mediante `ingest_records()` y re-ejecuta la generación con el corpus ampliado. Este mecanismo garantiza que el RAG pueda responder consultas sobre información no presente en el corpus crawlado originalmente.

## 8 Módulo de Búsqueda Web

### 8.1 Diseño y Activación Automática

El módulo de búsqueda web (`web_search.py`, `web_search_engine/`) implementa DuckDuckGo como mecanismo de ampliación automática del corpus local. La activación requiere el cumplimiento simultáneo de dos umbrales configurables:

- Número de documentos locales recuperados  $< \text{min\_local\_docs}$  (default: 5).
- Puntuación promedio de los resultados locales  $< \text{min\_score\_threshold}$  (default: 0,3).

La combinación de ambos criterios evita activaciones innecesarias: si hay pocos documentos pero con puntuación alta, o muchos documentos con puntuación baja, el corpus local se considera suficiente. El módulo también puede activarse directamente desde el RAGPipeline cuando el LLM indica insuficiencia de documentos.

### 8.2 Pipeline de Búsqueda

- Consulta a DuckDuckGo HTML (html.duckduckgo.com): se construye la URL de búsqueda con `quote_plus` y se parsea el HTML de resultados mediante tres patrones de expresiones regulares que cubren distintos formatos de respuesta de DDG, con un fallback de extracción de URLs genéricas.
- Descarga paralela: hasta 5 URLs concurrentes con `ThreadPoolExecutor` y `as_completed`, con timeout de 12 s por URL. Se descartan respuestas con `Content-Type` diferente de `text/html`.
- Extracción de contenido: `SimpleHTMLExtractor` extrae título, texto visible e idioma de cada página descargada, reutilizando el componente del `crawler_engine`.
- Persistencia: los registros se guardan en el JSONL principal (`webpages.jsonl`) para uso futuro, con `doc_id` secuencial, `crawl_date` y `score_hint` basado en posición en los resultados de DDG.

### 8.3 Integración con el Motor

Los documentos obtenidos se integran al motor mediante `GVSMSearchEngine.ingest_records()`, que actualiza: `self.records` (registros completos), `self.doc_tokens_by_id` (tokens), `self.doc_vectors` (vectores GVSM), estructuras BM25 (`term_doc_freq`, `doc_tf_body`, `doc_tf_title`, `doc_len`) y `ChromaDB` vía `upsert_documents()`. Tras la ingesta, IDF y el índice BM25 se recomputan completamente para reflejar los nuevos documentos.

| Aspecto            | Búsqueda Local (GVSM)        | Búsqueda Web (DDG)         |
|--------------------|------------------------------|----------------------------|
| Fuente de datos    | Corpus crawlado (1.361 docs) | DuckDuckGo (web abierta)   |
| Latencia           | < 1 s                        | 5 – 15 s                   |
| Activación         | Siempre                      | Automática (umbral dual)   |
| Persistencia       | JSONL indexado previamente   | Se agrega al JSONL         |
| Control de calidad | Alto (filtros del crawler)   | Moderado (extracción HTML) |
| Cobertura          | Dominio acotado y curado     | Web abierta ilimitada      |

## 9 Módulo de Interfaz Visual

### 9.1 Tecnología y Diseño

La interfaz se implementó como una Single Page Application (SPA) con React 18 y Vite 5 como bundler. Se comunica con el servidor Flask mediante peticiones HTTP asíncronas a los endpoints `/search` y `/rag`. El diseño visual emplea una paleta de colores neutra (azul pizarra `#406d8f` como acento principal, fondos grises suaves `#edf1f5` y `#f7f9fb`) y la tipografía Nunito Sans importada desde Google Fonts, optimizando la legibilidad de contenido técnico denso.

### 9.2 Componentes Principales

- TopNav: barra de navegación superior con el logotipo SRI, selector de idioma (ES/EN) con menú desplegable y banderas SVG, y botón de configuración. El cambio de idioma actualiza en tiempo real todos los textos y persiste la selección en `localStorage`.

- SearchPortal: componente central que integra el campo de búsqueda con icono SVG, los FilterPills de tipo, el panel RAG con fuentes clicables y la lista paginada de resultados.
- FilterPill: botones de filtro accesibles con atributo aria-pressed, soporte de teclado y transición visual de estado activo/inactivo.
- icons.jsx: iconografía SVG propia (búsqueda, carpeta, imagen, video, PDF, archivo, globo, engranaje, banderas GB y ES), garantizando independencia de librerías de íconos externas.

### 9.3 Flujo de Interacción

7. El usuario escribe una consulta en el campo de búsqueda y presiona BUSCAR o Enter.
8. La interfaz lanza en paralelo GET /search?query=...&top\_k=80 y GET /rag?query=...&top\_k=5 mediante Promise.allSettled, garantizando que un fallo de un endpoint no bloquee el otro.
9. Indicadores de estado se muestran de forma independiente: 'Buscando...' para resultados y 'Generando respuesta con RAG...' para el panel generativo.
10. El panel RAG se muestra en posición prominente (parte superior de los resultados) con la respuesta generativa, el contador de fuentes y las fuentes como pills clicables que abren el documento original.
11. Los resultados se muestran paginados (6 por página) con título clicable, snippet de 320 caracteres, dominio, fecha de crawl (solo la parte de fecha) y etiqueta de tipo de archivo.
12. El subtítulo animado rota cada 3 segundos entre 7 frases descriptivas del sistema con transición fade-out/fade-in de 280 ms.

### 9.4 Internacionalización

La interfaz soporta inglés y español completamente mediante el módulo i18n.js, que define el objeto UI\_COPY con todas las cadenas de texto en ambos idiomas, incluyendo funciones para mensajes parametrizados (getResultsCountText, getSearchingText, etc.). El idioma seleccionado se persiste en localStorage ('sri-language') y se refleja en el atributo lang del elemento html para compatibilidad con lectores de pantalla.

## 10 Módulo de Posicionamiento

### 10.1 Estrategia de Ranking

El módulo de posicionamiento determina el orden en que los resultados se presentan al usuario combinando múltiples señales. La estrategia prioriza relevancia semántica sobre popularidad, dado que en un dominio técnico la pertinencia del contenido es más valiosa que métricas de tráfico que no están disponibles en el corpus crawlado.

Los factores de posicionamiento se aplican en el siguiente orden:

- Puntuación de fusión GVSM + BM25 + ChromaDB (factor primario, rango [0,1]): calculada según la fórmula descrita en la Sección 5.4.
- Boost por coincidencia de subcadena: documentos cuyo título o texto contiene literalmente la consulta, pero que no quedaron en el top-k del ranking principal, reciben una puntuación adicional de 0,15 y se agregan al conjunto de candidatos.
- Rank de fuente como primer desempate: ante puntuaciones iguales, se prioriza el orden retornado originalmente por el motor (posición en el ranking inicial).

- Orden alfabético del título como último desempate: garantiza un ordenamiento estable, predecible y reproducible.

## 10.2 Posicionamiento Visual de la Información

El posicionamiento visual de los resultados en la interfaz sigue una jerarquía deliberada orientada a satisfacer distintos tipos de necesidades de información:

- Panel RAG en posición superior: la respuesta generativa ocupa el primer lugar visual, reconociendo que los usuarios con necesidades de información directas (factual queries) se benefician más de una respuesta sintetizada que de una lista de enlaces. Incluye el número de fuentes utilizadas y acceso directo a cada una.
- Lista de resultados paginada (6 por página): cada resultado muestra en orden visual: icono de tipo (izquierda), título clicable + snippet + metadatos (centro), etiqueta de tipo (derecha). Esta disposición en tres columnas facilita la exploración rápida.
- Snippet de 320 caracteres: suficiente para que el usuario evalúe la relevancia del documento sin necesidad de abrirlo, reduciendo clics innecesarios.
- Etiqueta de tipo de archivo: badge visual (PDF, Imagen, Video, Documento, Otro) derivado heurísticamente de la URL y el campo `source_type`, permitiendo al usuario filtrar mentalmente antes de hacer clic.
- Metadatos secundarios (dominio + fecha): presentados con menor contraste visual, disponibles para usuarios que los necesiten sin distraer la atención principal.

## 10.3 Decisiones de Diseño del Posicionamiento

- Relevancia sobre popularidad: en un dominio técnico, la pertinencia del contenido es más crítica que métricas de popularidad no disponibles en el corpus.
- Fusión normalizada: la normalización por máximo del conjunto de candidatos evita que diferencias de escala entre GVSM, BM25 y ChromaDB sesguen el ranking final.
- Frescura como metadato visual: la fecha de crawl se muestra al usuario para contexto, pero no afecta el ranking actual. Se identificó como mejora futura la incorporación de un factor de decay temporal.
- Paginación conservadora (6 resultados): reduce la carga cognitiva en cada página, aunque el backend retorna hasta 80 candidatos disponibles para paginación extendida.

# 11 Integración Completa del Sistema

## 11.1 Arquitectura por Capas

- Capa de datos: corpus JSONL (`data/extracted/webpages/webpages.jsonl`) + índices persistidos (Patricia Trie JSON, co-ocurrencia JSON, ChromaDB en `data/processed/`).
- Capa de lógica: servidor Flask (`scripts/main.py`) que instancia GVSMSearchEngine, RAGPipeline y WebSearchPipeline, exponiendo los endpoints `/search` y `/rag`.
- Capa de presentación: SPA React/Vite (`webapp/`) que consume la API REST, soporta internacionalización y es desplegable de forma independiente.

## 11.2 Flujo End-to-End

13. Usuario escribe consulta en la interfaz React.
14. Frontend lanza en paralelo `GET /search?query=...&top_k=80` y `GET /rag?query=...&top_k=5`.

15. Servidor Flask tokeniza la consulta con `Index.tokenize()` y ejecuta recuperación híbrida Trie + ChromaDB.
16. Ranking por fusión normalizada BM25 + GVSM + ChromaDB con boost de subcadena.
17. Evaluación de umbral web: si `local_count < 5` y `avg_score < 0,3`, activa `WebSearchPipeline`.
18. Respuesta `/search`: JSON con lista de documentos ordenados por score, incluyendo `doc_id`, título, snippet, url, `file_type`, `crawl_date`, domain, score y `source_type`.
19. En paralelo: `RAGPipeline` recupera top-5, construye prompt, llama a Cerebras LLM con retry exponencial.
20. Si LLM indica `sufficient = false`: `WebSearchPipeline` amplía corpus, `ingest_records()` actualiza el motor, re-generación.
21. Frontend renderiza panel RAG y lista paginada simultáneamente al recibir ambas respuestas.

### 11.3 Manejo de Errores y Resiliencia

- Flask CORS: cabecera `Access-Control-Allow-Origin: *` en todas las respuestas mediante `@app.after_request`.
- Retry exponencial Cerebras: 3 intentos con espera 1 s, 2 s, 4 s antes de activar el fallback extractivo.
- Fallback extractivo RAG: `ImprovedRAGGenerator` garantiza disponibilidad del módulo RAG ante fallos completos de la API de Cerebras.
- `Promise.allSettled` en frontend: un fallo de `/search` o `/rag` no bloquea la otra petición; cada una falla de forma independiente sin degradar la experiencia total.
- ChromaDB opcional: si el paquete `chromadb` no está instalado, el motor funciona únicamente con Trie + GVSM + BM25, degradando graciosamente la capa vectorial.
- Validación de `doc_id`: registros sin `doc_id` convertible a entero se ignoran en todos los módulos de indexación y ranking.

## 12 Despliegue con Docker

### 12.1 Arquitectura de Contenedores

El sistema se despliega mediante Docker Compose con dos servicios independientes definidos en `docker-compose.yml`:

- Servicio api: construido desde el Dockerfile raíz, basado en `python:3.12-slim`. Instala dependencias de `requirements.txt`, copia `scripts/` y `data/`, expone el puerto 5000 y ejecuta `python scripts/main.py serve`.
- Servicio web: construido desde `webapp/Dockerfile`, basado en `node:20-alpine`. Instala dependencias npm, expone el puerto 5173 y ejecuta `npm run dev` con binding a `0.0.0.0` para acceso desde el host.

El directorio `data/` se monta como volumen en el servicio api (`./data:/app/data`), permitiendo que el corpus, los índices y la base de datos ChromaDB persistan entre reinicios del contenedor sin necesidad de reconstruir la imagen.

### 12.2 Instrucciones de Despliegue

Requisitos previos: Docker Engine  $\geq 20.10$  y Docker Compose  $\geq 2.0$ .

Paso 1 — Construir las imágenes:

```
docker compose build
```

Paso 2 — Iniciar el servicio API (esperar hasta ver 'Sistema inicializado correctamente'):

```
docker compose up api
```

Paso 3 — En una nueva terminal, iniciar el frontend:

```
docker compose up web
```

Paso 4 — Acceder al sistema:

- Interfaz web: <http://localhost:5173>
- API REST: <http://localhost:5000/search?query=python>

Paso 5 — Detener todos los servicios:

```
docker compose down
```

## 12.3 Consideraciones de Producción

- El servicio web usa el servidor de desarrollo de Vite (npm run dev). Para producción se recomienda ejecutar npm run build y servir los archivos estáticos con nginx.
- La clave de API de Cerebras debe configurarse en el archivo .env antes de construir la imagen: CEREBRAS\_API\_KEY=csk-...
- El archivo .dockerignore excluye: .env, \_\_pycache\_\_, node\_modules, logs, .git y .DS\_Store para reducir el contexto de construcción.

## 13 Análisis Crítico del Sistema

### 13.1 Bondades

- Arquitectura modular y desacoplada: la separación en crawler\_engine, rag\_engine, web\_search\_engine y engine.py facilita el mantenimiento, la prueba y la extensión independiente de cada componente.
- Bilingüismo nativo: manejo coherente de documentos en inglés (80,21 %) y español (19,79 %) con detección de idioma en cascada, stopwords para ambos idiomas y corpus bilingüe integrado.
- Fusión de señales heterogéneas: la combinación normalizada de GVSM (semántico), BM25 (léxico) y ChromaDB (vectorial) produce un ranking más robusto y complementario que cualquier señal individual.
- Resiliencia del módulo RAG: el fallback extractivo garantiza disponibilidad del servicio ante fallos de la API de Cerebras, con degradación graceful y sin interrupciones para el usuario.
- Búsqueda web automática y transparente: ampliación del corpus sin intervención del usuario, con persistencia de los documentos obtenidos para consultas futuras.
- Despliegue dockerizado: los Dockerfiles y docker-compose.yml permiten reproducir el sistema en cualquier entorno compatible sin configuración manual de dependencias.
- Interfaz accesible y bilingüe: uso de atributos ARIA (aria-pressed, aria-label, aria-live), soporte completo de internacionalización y diseño responsivo para pantallas pequeñas.
- Carga incremental del Trie: sincronización con el JSONL al iniciar evita reconstruir el índice completo ante nuevos documentos, acelerando el tiempo de arranque.

### 13.2 Insuficiencias y Limitaciones

- Embeddings hash sin semántica real: los embeddings generados por proyección SHA-1 no capturan significado semántico, por lo que la búsqueda vectorial ChromaDB actúa más como una señal de diversificación que de recuperación semántica genuina.
- Trie no actualizado con documentos web: `ingest_records()` actualiza `self.records`, `doc_vectors` y BM25 pero no el Patricia Trie en memoria ni en disco, por lo que los documentos ingresados dinámicamente solo son recuperables vía ChromaDB o BM25, no por búsqueda Trie.
- Escalabilidad limitada: el pre-cómputo de vectores GVSM para todos los documentos en memoria y la reconstrucción completa del índice BM25 ante nuevas ingestas son ineficientes para corpus de más de 100.000 documentos.
- Ausencia de evaluación formal: el sistema carece de un conjunto de consultas de prueba con relevancia anotada, lo que impide medir Precision, Recall, NDCG@k o MRR de forma rigurosa y comparar configuraciones.
- Sin detección explícita del idioma de la consulta: el sistema no detecta el idioma del usuario para adaptar el modelo spaCy o el idioma del prompt RAG; responde en el idioma de los documentos recuperados, no necesariamente en el de la consulta.
- Scraping limitado a HTML estático: el extractor HTML puede fallar en páginas con renderizado JavaScript (React, Angular, Vue en modo SPA), retornando contenido vacío o incompleto para sitios modernos.
- Servidor web de desarrollo en Docker: el servicio web usa `npm run dev` (servidor Vite de desarrollo) incluso en el contenedor Docker, lo cual no es apropiado para entornos de producción real.

### 13.3 Posibles Soluciones a las Deficiencias

- Embeddings semánticos: integrar sentence-transformers (all-MiniLM-L6-v2 u otro modelo ligero) en `ChromaVectorStore._embed_text()` manteniendo la misma interfaz pública, sin cambios en el resto del sistema.
- Trie dinámico con ingesta web: modificar `ingest_records()` para llamar a `self.trie.insert(token, doc_id)` por cada token de los documentos ingresados y ejecutar `self.trie.save()` al finalizar, garantizando recuperación Trie completa.
- BM25 incremental: mantener estructuras delta separadas (`delta_tf_body`, `delta_doc_len`) para documentos nuevos y fusionarlas en cada búsqueda, evitando la reconstrucción completa del índice BM25.
- Conjunto de evaluación anotado: construir manualmente 50-100 consultas técnicas con relevancia anotada (relevant/non-relevant por documento) e implementar métricas MAP, NDCG@10 y MRR para comparar configuraciones de ranking.
- Detección de idioma de consulta: aplicar `infer_language_from_text()` o `langdetect` a la consulta y seleccionar el modelo spaCy apropiado (`en_core_web_sm` / `es_core_news_sm`) dinámicamente.
- Scraping con Playwright o Selenium: usar un navegador headless que espere la hidratación del DOM para páginas con renderizado JavaScript antes de extraer contenido.
- Build de producción en Docker: reemplazar el servicio web con un stage multi-etapa que ejecute `npm run build` y sirva los archivos estáticos con `nginx:alpine`.

## 14 Guía de Uso del Sistema

### 14.1 Requisitos del Sistema

Para ejecución sin Docker:

- Python 3.8+ con las dependencias de requirements.txt instaladas (Flask, spaCy, numpy, scipy, chromadb, cerebras-cloud-sdk).
- Node.js >= 16 y npm para el frontend React/Vite.
- Conexión a internet para el módulo de búsqueda web DuckDuckGo y la API de Cerebras.

Para ejecución con Docker:

- Docker Engine >= 20.10 y Docker Compose >= 2.0.
- Conexión a internet para descargar las imágenes base y para los módulos de búsqueda web y RAG.

### 14.2 Instalación sin Docker

22. Clonar el repositorio: `git clone <url-del-repositorio> && cd <nombre-del-repositorio>`.
23. Crear entorno virtual: `python3 -m venv .venv && source .venv/bin/activate` (Windows: `.venv\Scripts\activate`).
24. Instalar dependencias Python: `pip install -r requirements.txt`.
25. Instalar dependencias del frontend: `cd webapp && npm install && cd ..`
26. Configurar la clave Cerebras creando el archivo `.env` en la raíz del repositorio con el contenido: `CEREBRAS_API_KEY=csk-pv89mjp25xr4p3cyhcy4584rrpyfxrchemc4k5x5kmewed9k`

### 14.3 Ejecución sin Docker

27. (Opcional) Crawling del corpus: `python3 scripts/tech_crawler.py --max-pages 500 --max-depth 2 --seeds-file scripts/tech_seeds.txt`
28. Iniciar servidor backend (esperar mensaje 'Sistema inicializado correctamente'):  
`python3 scripts/main.py serve`
29. En otra terminal, iniciar el frontend: `cd webapp && npm run dev`
30. Abrir `http://localhost:5173` en el navegador.

### 14.4 Ejecución con Docker

31. Construir imágenes: `docker compose build`
32. Iniciar API: `docker compose up api` (esperar inicialización completa en los logs).
33. En otra terminal, iniciar frontend: `docker compose up web`
34. Acceder a `http://localhost:5173`. Para detener: `docker compose down`

### 14.5 Uso de la Interfaz

35. Escribir una consulta técnica en el campo central (ej.: 'python machine learning', 'kubernetes deployment', 'React hooks performance').
36. Presionar BUSCAR o Enter. La interfaz cargará simultáneamente el panel RAG y la lista de resultados.
37. Revisar la respuesta generativa en el panel RAG (parte superior). Las fuentes citadas aparecen como pills azules clicables.

38. Explorar la lista de resultados: hacer clic en el título de cualquier resultado para abrir el documento original en una nueva pestaña.
39. Usar los botones < y > para paginar entre resultados (6 por página).
40. Cambiar el idioma de la interfaz (ES/EN) desde el selector en la barra de navegación superior.

## Referencias

- [1] Wong, S.K.M., Ziarko, W., Wong, P.C.N. (1985). Generalized vector space model in information retrieval. In Proceedings of the 8th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 18–25. ACM, New York.
- [2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems 33 (NeurIPS 2020).
- [3] Robertson, S., Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389.
- [4] Malkov, Y.A., Yashunin, D.A. (2020). Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824–836.
- [5] Manning, C.D., Raghavan, P., Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.
- [6] Cerebras Systems (2024). Cerebras Cloud SDK Documentation. <https://docs.cerebras.ai>
- [7] Chroma (2024). Chroma — the AI-native open-source embedding database. <https://docs.trychroma.com>
- [8] Meta AI (2024). Llama 3: Open Foundation and Fine-Tuned Chat Models. <https://ai.meta.com/llama/>
- [9] Honnibal, M., Montani, I., Van Landeghem, S., Boyd, A. (2020). spaCy: Industrial-strength Natural Language Processing in Python. Zenodo. <https://doi.org/10.5281/zenodo.1212303>